ARCHITECTURE OVERVIEW · V0.0.1 · 2026

JARVIS

A local-first AI assistant for Windows
with strict approvals and a replaceable model gateway.

GITHUB.COM/S-FETAH/MYJARVIS

ELECTRON · REACT · TYPESCRIPT

One assistant that does real work — safely

Not a chat toy: an agent that plans, acts with permission, and proves the result.

The experience

- Ask in plain language for **multi-step, practical work**: research, coding, summaries, PDFs, downloads, dev servers.
- Jarvis **shows its plan and live progress**, asks before anything sensitive, and **verifies outcomes** — a file exists, a server listens, a command exits clean.
- Report comes back with links to everything it produced.

Why local-first

- Files, credentials, and tool execution **stay on your machine**.
- The model gateway runs locally; providers are called through it — never hardcoded, never locked in.
- Autonomy is a **spectrum under your control**: every side effect is classified, logged, and gated where it matters.

> *"Reason about a task, show progress, use approved tools, verify the outcome, report clearly."*

First-release scope — and hard non-goals

Balanced coverage across browsers, code, and documents; never unrestricted control.

Deliverables

- Chat UI with streaming, task progress, approvals, and history
- Browser search + structured automation (Playwright-class, not mouse simulation)
- Approved-folder file read/write + summarisation
- PDF creation **with output verification**
- Project tooling: tests, dev servers, process inspection + approved stops
- Image generation via a supported provider

Non-goals / guardrails

- No cloud-hosted agent — sensitive actions stay local
- No unrestricted shell or computer control
- No extracting or pooling subscription quotas (Codex plans are not API quota)
- Web content is **untrusted input**, never authority over the user's request
- General desktop UI automation deferred — too fragile for v1

Evidence-driven, policy-first, incremental

Every layer of the design was validated by a spike before it was committed to.

Prove, then expand

One model, one well-bounded tool, one verified loop first. Toolset and model routing grow in small, verifiable steps — never all at once.

Policy before features

The permission engine and approval binding were designed and unit-tested **before** the agent loop that needs them (ADR-004).

Benchmark, don't assume

T-001 validated the gateway's real latency and failure modes; T-002 benchmarked an existing agent to decide build-vs-embed with evidence.

Build custom, embed nothing

T-002 showed Open Interpreter was not embeddable and its default gates too weak — so Jarvis runs a **thin purpose-built orchestrator** in its trusted process (ADR-002).

Replaceable gateway

All model traffic flows through one `IModelGateway` adapter. FreeLLMAPI is the first backend; Ollama, LM Studio, or direct APIs can replace it without touching agent logic (ADR-003).

04 · STACK

Proven desktop stack, zero exotic dependencies

Verified on Node 24 · Electron 34 · React 19 · Windows — typecheck clean, build passing.

Application

`Electron 34` `React 19` `Vite 6` `TypeScript 5.7` `Tailwind 3` `electron-vite`

Three-window shell: main UI, quick-chat drawer, 3D mascot.

Agent & state

`Custom TS orchestrator` `Zod schemas` `sql.js (SQLite)` `safeStorage/DPAPI`

Task state is model-independent — a fallback model can continue a task.

Model gateway

`FreeLLMAPI v0.16.7` `OpenAI-compatible API` `127.0.0.1:31415`

279 models behind one local endpoint; streaming + tool calls validated.

Visual & UX

`Three.js (mascot)` `lucide-react` `react-markdown`

Quality

`Vitest (22/22)` `tsc --noEmit` `eslint + prettier` `security-scan script`

Planned tools

`Playwright (browser)` `PDF pipeline` `image provider`

Process model — privilege stops at one boundary

The renderer can never touch Node, the disk, or the network directly (ADR-002).

Renderer — unprivileged

React UI: chat, plan, timeline, approval modal, mascot.

`contextIsolation: ON` `nodeIntegration: OFF` `sandbox: ON` `strict CSP`

▼

Preload bridge — typed, allowlisted

Only declared request/response and event contracts cross. No raw streams, file handles, or spawn.

`contextBridge.exposeInMainWorld` `zod-validated messages` `sender verified`

▼

Main — the only trusted process

Agent orchestrator · policy engine (HMAC approvals) · tool executors · gateway client · SQLite history.

`powershell -NoProfile` `spawn(shell:false)` `minimal env` `owned-process tracking`

▼

Windows PC — approved scope only

Approved workspaces, selected projects, local gateway. Reads outside scope require approval; writes outside scope always do.

`C:\… approved folders` `127.0.0.1:31415 gateway` `DPAPI credential store`

Audited, remediated, regression-tested

2026-10-23 audit found 11 issues (SEC-001…011) — all P0/P1 findings remediated with 22 automated tests.

Hardened boundaries

- **Command injection eliminated** — parameterized `spawn(shell:false)`, strict URL protocol checks (SEC-001)
- **HMAC approval verification wired** into the runtime resolver, anti-tamper tested (SEC-002)
- **Credentials encrypted** via Electron safeStorage (DPAPI) in SQLite; renderer only sees a `hasApiKey` flag (SEC-003)

Window & filesystem defense

- **Sandbox + context isolation** enforced; navigation guards and window-open handler (SEC-004/005/011)
- **Symlink traversal blocked** — realpath resolution on every tool path (SEC-006)
- **Extension gating** — `.bat .cmd .ps1 .exe` treated as high-risk (SEC-007)
- **Atomic persistence** — no torn writes on crash (SEC-010)

11
FINDINGS REMEDIATED

22/22
TESTS PASSING

4
ADRS GOVERNING DESIGN

0
SECRETS IN REPO OR RENDERER

Six categories, cryptographically bound approvals

Stricter than the benchmarked competitor default: outside reads are gated too, not just writes.

CATEGORY	SCOPE	DEFAULT	UI
`read_in`	Files inside an approved workspace	**Allowed**	Live timeline
`read_out`	Anything outside workspaces (Desktop, profile)	**Approval**	Modal + full path
`write_in`	Create/modify inside workspace	**New files OK**	Timeline + diff
`write_out`	Create/modify outside workspaces	**Approval**	Modal + content preview
`process`	Child processes, dev servers, scripts	**Approval**	Modal: exe + args + cwd
`network`	Search, navigation, downloads	**Read-only OK**	Timeline + URLs + bytes

Approval binding

Every approval = `HMAC-SHA256(action ‖ workspace ‖ args-hash, session-secret)`. **Single-use, expires in 5 minutes.** Change one argument — the token fails.

Denial semantics

Deny permanently invalidates the token. The runtime **never silently retries** a denied action — it halts the branch and asks for new guidance.

One adapter, deterministic models, honest failures

Designed directly from measured gateway behavior (T-001), not vendor claims.

Deterministic routing

- Profiles pin **concrete model IDs** (fast / reasoning / coding / vision) fetched live from `GET /v1/models`
- Gateway `"auto"` routing measured **non-deterministic** — reserved as fallback only

Failure semantics

- Local gateway down (`ECONNREFUSED`) vs upstream error (`502/503/504`) are **distinct signals**
- Transient 5xx → **exactly one retry** after 3 s, exponential backoff
- Never auto-retries side effects; inference requests only

Latency instrumentation

TTFT clock starts at `fetch()` dispatch — capturing connect + queue (measured gap: ~3.3 s hidden in connection). Telemetry logs pinned model, routed model, TTFT, total, tokens.

Credential boundary

Keys live only in env vars or DPAPI-encrypted storage — never in SQLite plaintext, never in Git, never in the renderer, never in event streams shown to the model.

09 · EVIDENCE

The spikes behind every design choice

T-001 validated the gateway; T-002 benchmarked Open Interpreter 0.0.56 in a disposable sandbox.

278
MODELS LISTED · 78 MS

1.4s
BEST CHAT LATENCY

64s
WORST TURN (TOOL CALL)

502
TRANSIENT UPSTREAM FLAKE

T-001 — gateway validation

- Authenticated model list, chat, streaming, and a **non-executed tool call** all passed
- Streaming TTFT must be measured from dispatch, not body-open
- → became ADR-003 (pin models, one bounded retry, honest 5xx)

T-002 — competitor benchmark

- Outside reads and inside writes executed with **zero approval**; denial of outside writes was honored
- Shells inherited the host profile (errors every turn); model picker silently bypassed the gateway
- → became ADR-002/004: custom orchestrator, stricter gates, clean shells

Three modes — latency without recklessness

Racing is a hedge against slow routes; it never applies to side effects.

Normal

One pinned model per profile. The default for everything. Predictable routing, clean telemetry, lowest cost.

Racing

For **eligible read-only requests only**: fire a bounded race across profiles, take the first valid response, cancel the losers. Never for tools, downloads, mutations, or sensitive prompts.

Multi-agent

Separate **visible model roles** (researcher, coder, reviewer) — each policy-bound. Not unrestricted parallel tool control; one shared approval timeline.

Why this exists

Measured spread on the same gateway: **1.4 s to 64 s** for equivalent work. Hedging read-only turns with a cancel-losers race turns worst-case waits into typical-case ones — without ever racing an action that touches the system.

Request lifecycle — one turn, end to end

Every turn walks the same policy-checked path; the user sees each step live.

1. **Prompt** arrives in the renderer; the typed bridge forwards it to main.
2. **Plan** — the orchestrator drafts steps and required tool categories.
3. **Policy classify** — each intended action is mapped to read/write/process/network tiers.
4. **Gateway call** — pinned model, streaming deltas; TTFT + total recorded.
5. **Approval gate** — gated actions raise an HMAC-bound modal; deny halts the branch.
1. **Execute sanitized** — `-NoProfile`, minimal env, parameterized spawn, scoped paths.
2. **Verify outcome** — file exists, server listens, exit code checks. Claims are not enough.
3. **Feed results back** — verified outputs return to the model for the next step.
4. **Timeline + history** — every event lands in the activity timeline and SQLite audit store.
5. **Report** — summary with links to produced artifacts; failures surface honestly.

v0.0.1 — a remediated, working baseline

Phase 0 closed; Phase 1 foundation and Phase 2 agent loop are in flight on GitHub.

Surface A — mascot

Transparent 3D owl (Three.js) with procedural idle/thinking/speaking states; click-through, 60 fps tracking.

Surface B — quick chat

Compact drawer from the mascot or hotkey for fast questions without the full window.

Surface C — main UI

Full shell: persistent chat history, new chat, Free Models Hub, chain-of-thought cards, rate-limit recovery.

Working now

- Model gateway client: streaming, bounded retry, thinking-block extraction
- Policy engine wired and verified at runtime (HMAC)
- Read-only agent loop + tool engine (Phase 2 commit)
- 22/22 tests · typecheck + production build clean

Next up (Phase 2)

- Playwright browser automation execution loop
- PDF generation + verification pipeline
- Active task dispatcher + streamed event log (T-005)
- Full write-tool integration behind approvals

Seven phases, one verifiable step at a time

A phase closes only when its plan's exit criteria are met — no exceptions.

P0	**Discovery & spike**	Gateway validated · agent benchmarked · ADR-001…004 recorded — **complete 2026-10-22**
P1	**Secure foundation** NOW	Electron shell, secure IPC, policy engine, persistence — scaffold + audit remediation done; T-005 event streams in progress
P2	**Read-only agent loop**	Bounded research and workspace tools — loop committed; browser + PDF tools landing
P3	**Documents & files**	Research-to-PDF vertical slice with output verification
P4	**Coding workflow**	Approved, tracked dev-server and test orchestration
P5	**Robustness**	Failure, cancellation, restart, and prompt-injection test passes
P6	**Windows integration**	Tray, settings, clean install/upgrade validation, installer
P7	**Expansion**	Image generation, richer automation, optional voice — only after MVP acceptance

The target experience

What "done" looks like when the phases land.

"Research this and make me a PDF"

Jarvis searches the web with structured automation, cross-checks sources, drafts the document, generates the PDF through an approved write, **opens it to verify it renders**, and hands you the link — with every step visible in the timeline.

"Get this project running"

It inspects the codebase, runs the test suite, and starts the dev server — each process launch approved once and tracked. It reports the listening port and only ever stops **processes it started**, after asking.

Always-on, low-friction

Tray-resident with the mascot and quick-chat drawer; the main window for real work. Model profiles switch per task; Racing mode keeps read-only answers fast; rate limits recover gracefully with your guidance.

Trustworthy by construction

Every outside-scope action stops at a modal bound to the exact arguments. Denials are final. The full audit trail — plans, commands, outputs, approvals — lives locally in SQLite, exportable and inspectable.

IN SUMMARY

Local-first. Policy-first. Evidence-driven. Replaceable at every boundary.

A custom Electron orchestrator over a validated local gateway, with cryptographic approvals, sanitized execution, and verified outcomes — built one proven loop at a time.